FADA: Few-Shot Domain Adaptation via Dynamics Alignment for Humanoid Control

Angchen Xie* Nikhil Sobanbabu* Ishayu Shikhare Alan Wang
Max Simchowitz Guanya Shi

Carnegie Mellon University *Equal contribution

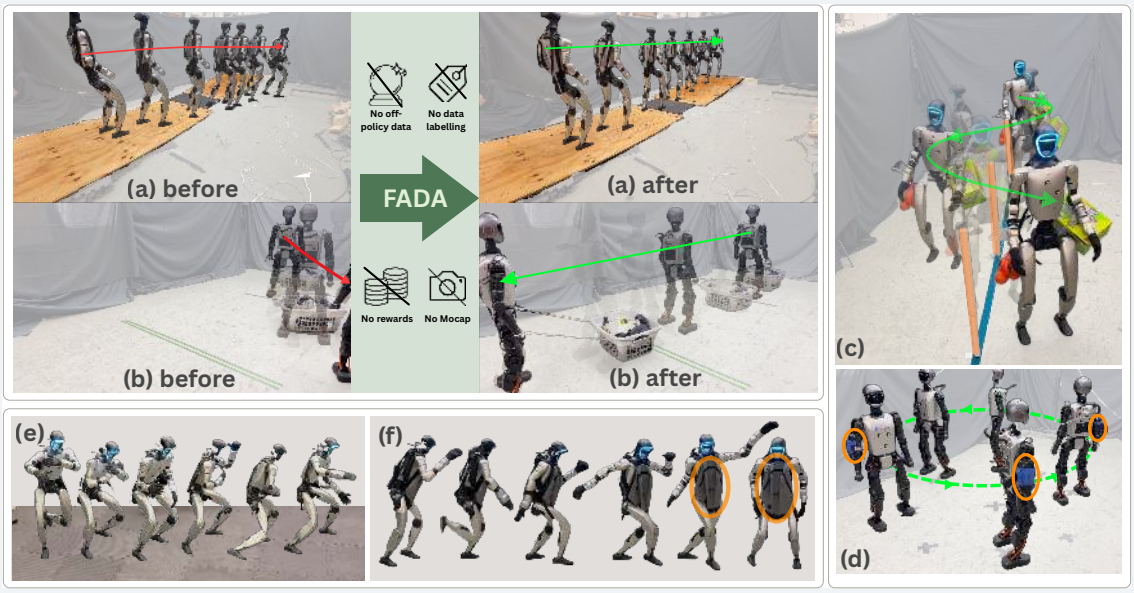

Figure 1: FADA enables high-precision whole-body skills through dynamics alignment. Through few-shot adaptation, humanoid robots can stably execute diverse real-world tasks that fail under zero-shot transfer. (a) and (b) illustrate the adaptation effect: Only after adaptation is Unitree G1 able to precisely track a line on a slope, and Booster T1 able to pull a 6 kg laundry basket across the finish line. (c)–(f) show additional adapted deployments: (c) G1 weaving through poles with asymmetric 3 kg groceries, (d) T1 tracking a circular path with an asymmetric 1 kg arm payload, (e) G1 executing Kung Fu motions on soft mats, and (f) G1 dancing with a 3.2 kg front payload.

Abstract

High-precision humanoid control is limited by target-domain dynamics mismatch, where the same control objective can induce different realized motions under changes in terrain, payload, or actuator response. Existing methods either pursue zero-shot transfer through domain randomization or in-context adaptation without target-domain specialization, or require heavy adaptation pipelines that leverage target-domain data, such as model calibration, residual learning, or policy retraining. In this paper, we present **FADA** (**F**ew-Shot Domain **A**daptation via **D**ynamics **A**lignment), a three-stage Planner–Inverse Dynamics Model (Planner–IDM) framework for few-shot adaptation in humanoid control. **FADA** first trains an oracle policy with privileged information and then distills the oracle behavior into a deployable Planner–IDM student through DAgger. At deployment, **FADA** freezes the planner and finetunes only the IDM using approximately 2 minutes of target-domain rollouts with standard supervised learning. Rather than requiring optimal demonstrations or rewards, **FADA** uses the paired actions and observations that are observed during these rollouts as supervision, aligning the IDM's action generation with target-domain dynamics. Experiments show that **FADA** outperforms both in-context and end-to-end adaptation baselines, improving task performance under dynamics shifts and enabling real humanoid robots to execute diverse high-precision whole-body tasks. Implementation details and qualitative hardware rollout videos are available at https://lecar-lab.github.io/FADA-humanoid/.

1 Introduction

Humanoid robots must perform precise simultaneous locomotion and manipulation in complex environments. Recent reinforcement learning methods have demonstrated agile and robust locomotion [Gu et al., 2024], whole-body tracking [He et al., 2025, 2024b], teleoperation [He et al., 2024a, Fu et al., 2024], and loco-manipulation [Zhao et al., 2025, Zhang et al., 2025a]. These advances rely heavily on simulation, where policies can safely experience falls, impacts, contacts, and task perturbations at scale. However, despite success in simulation, these methods still struggle with *precise* control on real hardware. When deployment conditions differ from training, the same control intent may no longer produce the same body motion. This gap arises from a dynamics mismatch between simulation and the target domain, including differences in terrain, payload, and actuator response. For humanoids, these discrepancies are amplified by tightly coupled whole-body dynamics: small errors in foot placement, contact timing, or force delivery can lead to posture drift, degraded tracking, or instability. Such failures often do not indicate an unreasonable high-level command; rather, they arise when the motion or control output no longer induces the intended physical response in the target domain.

Existing approaches for improving target-domain deployment broadly fall into two categories, as summarized in Figure 2. The first category seeks zero-shot robustness or in-context adaptation, including domain randomization [Gu et al., 2024, Bohlinger and Peters, 2025, Cha et al., 2025] and history-conditioned adaptation [Kumar et al., 2021a, Nahrendra et al., 2023, Long et al., 2023, Liu et al., 2025]. These methods make deployment more robust, but do not update policy weights with target-domain data. They therefore remain conservative under the target condition. The second category uses target-domain data for adaptation, including system identification [Ren et al., 2023, Sobanbabu et al., 2025, Chen et al., 2022], residual dynamics learning [Lee et al., 2025, He et al., 2025], and policy finetuning [Smith et al., 2021, Hu et al., 2025, Huang et al., 2026]. These methods can specialize to the target domain, but typically require either a predefined model to fit, new demonstrations to imitate, or a computable target-domain reward for policy optimization. In this work, we ask:

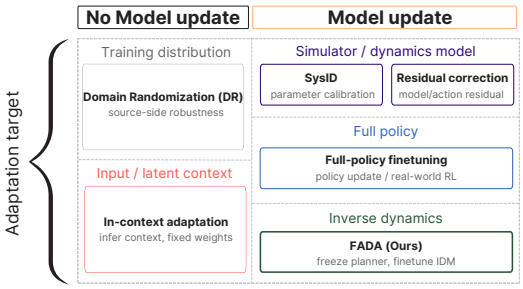

Figure 2: Adaptation taxonomy. Existing approaches differ in whether they use target rollouts for model updates and which component they update. FADA updates the IDM with target-domain rollouts.

> *Is it possible to achieve few-shot target-domain adaptation for precise humanoid control* ***without rewards or accurate prior models****?*

We answer in the affirmative by introducing FADA: a three-stage Planner–Inverse Dynamics Model (Planner–IDM) framework for few-shot domain adaptation via dynamics alignment. FADA first trains a privileged oracle policy in simulation, then distills the oracle behavior into a deployable Planner–IDM student through DAgger. The planner predicts short-horizon proprioceptive intent, and the IDM converts this intent together with recent execution history into actions. At deployment, FADA collects a small number of target-domain rollouts, freezes the planner, and finetunes only the IDM with supervised learning. This aligns the IDM with target-domain dynamics while avoiding simulator fitting, real-world reinforcement learning, and full-policy updates. We evaluate FADA in cross-simulator transfer and real-hardware deployment. Our experiments cover IsaacSim-to-MuJoCo transfer and deployment on Unitree G1 and Booster T1. As shown in Figure 1, after few-shot IDM finetuning, the Planner–IDM controller reliably performs high-precision whole-body tasks on hardware, including locomotion, whole-body tracking, and loco-manipulation. Results

show that FADA improves target-domain performance in both sim-to-sim and sim-to-real settings, demonstrating that supervised IDM finetuning from only a few target-domain rollouts can enable effective adaptation under dynamics shifts.

2 Related Work

Domain adaptation for humanoid whole-body control is commonly studied as a sim-to-real transfer problem. As summarized in Figure 2, existing methods can be organized by whether they use target-domain data to update the policy, and by which component they adapt: the source training distribution, the inferred deployment-time context, the simulator or dynamics model, the full policy, or the deployed controller.

2.1 Source-Side Robustness: Domain Randomization and Simulator Alignment

A common strategy is to absorb the sim-to-real mismatch before deployment. **Domain randomization** trains policies over distributions of physical and sensing parameters, such as mass, friction, actuator response, latency, and observation noise Gu et al. [2024], Fu et al. [2024], Cheng et al. [2024]. It is now standard in humanoid locomotion and whole-body control He et al. [2024b,a], Xie et al. [2025], Seo et al. [2025], with recent variants randomizing actuator dynamics, embodiments, or large-scale multi-skill curricula Cha et al. [2025], Bohlinger and Peters [2025], Sleiman et al. [2026], Wang et al. [2026]. However, broad randomization often yields an overly conservative policy rather than one specialized to the deployed system Da et al. [2025]. **Simulator alignment** instead uses target-domain data to align the source simulator or a dynamics model through system identification, residual dynamics, or learned correction models Ren et al. [2023], Sobanbabu et al. [2025], He et al. [2025], Krishna et al. [2025], Lee et al. [2025]. These methods provide target-specific corrections, but require simulator fitting and are limited by the chosen parameterization or correction model. FADA bypasses simulator refitting and adapts the execution module directly from target rollouts while preserving the source policy interface.

2.2 Inference-Time In-Context Adaptation

A second class of methods adapts at deployment by changing the policy input while keeping policy parameters fixed, corresponding to the "*input / latent context*" branch in Figure 2. RMA-style teacher–student methods train a privileged teacher and distill its latent into a deployable history encoder Kumar et al. [2021a], with extensions to bipedal hardware, humanoid locomotion, dynamic-load adaptation, and uncertainty-aware control Kumar et al. [2022], Cui et al. [2025], Chang et al. [2025], Kumar et al. [2021b]. Related world-model approaches replace privileged supervision with self-supervised prediction of terrain, proprioception, future states, or state-action evolution Nahrendra et al. [2023], Long et al. [2023], Liu et al. [2025], Lyu et al. [2025], Zhang et al. [2025b], Xue et al. [2026], Sun et al. [2025], Huang et al. [2025]. These methods adapt by updating the inferred context while keeping the action policy fixed. As a result, they can only exploit target-domain information through a source-trained mapping from context to actions. In contrast, FADA freezes the planner but finetunes the inverse-dynamics module, allowing target rollouts to directly adapt how planned motions are executed.

2.3 Target-Domain Learning: Finetuning and Model Adaptation

A third class of methods uses target-domain rollouts to update the deployed system after transfer. Residual methods keep a nominal policy fixed and learn an additive action or dynamics correc-

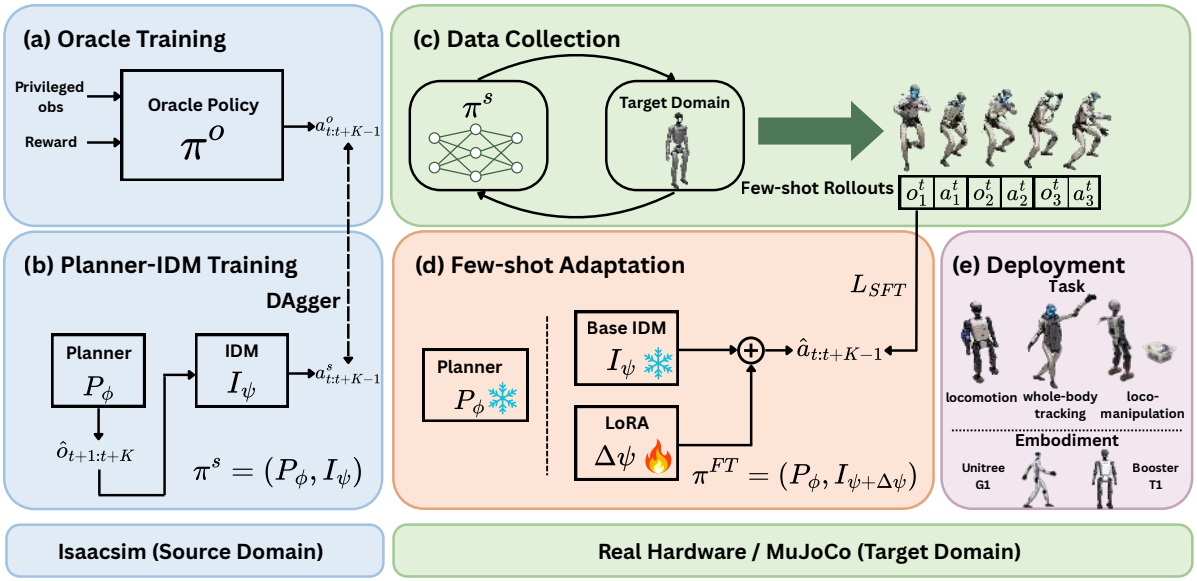

Figure 3: Overview of FADA. FADA first trains a privileged oracle policy in the source simulator, then distills it into a deployable Planner–IDM student through DAgger-style supervision. The planner predicts short-horizon future proprioception from the task command and observation history, while the IDM maps this future to actions. During target adaptation, FADA freezes the planner and finetunes only the IDM using a few target-domain rollouts.

tion Zhao et al. [2025], He et al. [2025], Cheng et al. [2025], but they act at the final action or model level and treat the policy internals as a black box. Policy finetuning and sim-and-real co-training update the policy itself using target-domain experience, such as real-world RL, human corrections, robot-assisted resets, hybrid online-offline RL, or mixed sim-real training Smith et al. [2021], Jiang et al. [2024], Hu et al. [2025], Lei et al. [2024], Jones et al. [2025], Maddukuri et al. [2025], Lei et al. [2026]. These methods can recover performance, but full-policy updates are data-intensive and may entangle command interpretation with low-level execution. World-model and planning-based methods adapt a learned dynamics model and use it for planning or policy optimization Levy et al. [2026], Li et al. [2025, 2026], but still require optimization through the learned model at deployment. In contrast, FADA adapts the execution module inside a factorized policy, requiring no target reward, privileged labels, simulator recalibration, online RL, or MPC-style planning.

3 Problem Setup

We study few-shot adaptation for a fixed task under changing deployment dynamics. Let $\mathcal{T}$ denote a task, such as whole-body tracking, payload locomotion, or loco-manipulation, and let ξ denote a deployment condition that captures terrain contact, payload, and sensing noise. For each task and condition, we consider an MDP $\mathcal{M}_{\mathcal{T},\xi} = (\mathcal{S}, \mathcal{O}, \mathcal{A}, p_\xi, r_\mathcal{T})$ where $s_t \in \mathcal{S}$ is the full state, $o_t \in \mathcal{O}$ is the proprioceptive observation available at deployment, $a_t \in \mathcal{A}$ is the action, and $c_t \in \mathcal{C}$ denotes the task command (e.g., desired velocity or reference motion). The transition dynamics are given by p_ξ and the task objective by $r_\mathcal{T}$.

Source-domain training. For each task $\mathcal{T}$, source training is performed over a distribution of simulated deployment conditions $\xi \sim \Omega_{\text{src}}$. We assume access to a standard privileged-teacher pipeline: a privileged oracle $\pi^o(s_t, c_t) \mapsto a_t^\star$ is trained in simulation using task rewards and privileged state information, and a student policy is trained from DAgger-style rollouts. The student receives only

proprioceptive observation history, action history, and task commands,

$$\pi^s(\mathcal{O}_t^H, \mathcal{A}_t^H, c_t) \mapsto a_t, \quad \mathcal{O}_t^H = (o_{t-H+1}, \ldots, o_t), \quad \mathcal{A}_t^H = (a_{t-H}, \ldots, a_{t-1}). \tag{3.1}$$

Thus, source training has access to privileged oracle actions as labels and DAgger-style rollouts of proprioceptive observations, commands, and executed actions.

Few-shot target deployment. At deployment, the source-trained student is evaluated under a fixed target condition ξ^{tgt}, such as a new simulator, hardware platform, payload, terrain, or contact regime. The task remains unchanged, but the transition dynamics may differ from those seen during source training. After zero-shot deployment, we collect a small set of target rollouts

$$\mathcal{D}_{\text{tgt}} = \{\tau_i\}_{i=1}^{N_{\text{roll}}}, \qquad \tau_i = \{(o_t, c_t, a_t)\}_{t=0}^{T_i}. \tag{3.2}$$

Unlike source training, which may have privileged information, target deployment has access only to these rollouts of proprioceptive observations, commands, and executed actions. The adaptation problem is to improve the source-trained student under ξ^{tgt} using only these rollouts, without target-domain rewards, privileged information, expert demonstrations, or simulator fitting.

4 FADA: Few-Shot Adaptation via Dynamics Alignment

Given the source-domain data and few-shot target rollouts defined in Section 3, FADA is built on a simple observation: under target-domain dynamics shifts, the task intention often remains meaningful, but the action required to realize it can change substantially. For example, changes in payload or terrain contact may not change what the robot should do, but they can change how the robot must act to produce the intended motion. This suggests adapting the execution component rather than relearning the entire policy; Section 5.6 validates this separation directly in a controlled setting. To make this distinction explicit, FADA factorizes the student policy into the Planner–Inverse Dynamics Model (IDM) interface shown in Figure 4: a planner that maps task commands and recent observations to a short-horizon proprioceptive intent, and an IDM that maps this intent and recent execution history to an action chunk. The overall training, adaptation, and deployment pipeline is summarized in Figure 3.

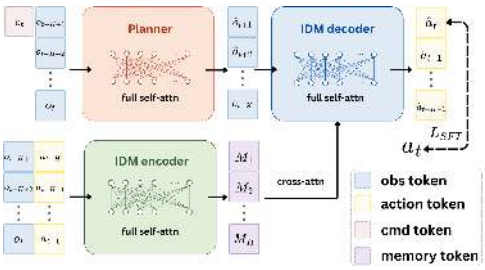

Figure 4: Planner–IDM interface. The planner predicts proprioceptive intent, and the IDM maps intent and execution history to an action chunk.

We instantiate the deployable student policy π^s as a Planner–IDM factorization, $\pi^s = (P_\phi, I_\psi)$. At time t, the planner P_ϕ predicts a K-step future proprioceptive sequence $\hat{Y}_t^K = (\hat{o}_{t+1}, \ldots, \hat{o}_{t+K})$, and the IDM I_ψ maps this predicted future, together with recent observation-action history, to an action chunk $\hat{U}_t^K = (\hat{a}_t, \ldots, \hat{a}_{t+K-1})$. The factorized student policy is

$$\hat{Y}_t^K = P_\phi(\mathcal{O}_t^H, c_t), \qquad \hat{U}_t^K = I_\psi(\mathcal{O}_t^H, \mathcal{A}_t^H, \hat{Y}_t^K), \qquad \hat{a}_t = \Pi_1(\hat{U}_t^K), \tag{4.1}$$

where Π_1 selects the first action in the chunk. Although the IDM outputs a full action chunk, deployment follows a receding-horizon interface: only $\hat{a}_t$ is executed before the planner and IDM are queried again. For source or target rollouts, the same notation applies to realized windows: $Y_{t,\text{exec}}^K = (o_{t+1}, \ldots, o_{t+K})$ and $U_{t,\text{exec}}^K = (a_t, \ldots, a_{t+K-1})$. These rollout windows provide inverse-dynamics supervision from the robot's own execution, even when the rollout is not task-optimal.

4.1 Learning the Planner–IDM Interface

Source training learns two coupled components: an IDM that maps realized proprioceptive futures to the actions executed over the same rollout segment, and a planner that predicts futures that are actionable through the IDM.

IDM learning: realized futures as inverse-dynamics supervision. Given a recent observation-action history and the future proprioceptive sequence observed over the same rollout window, the IDM predicts a K-step action chunk. Let $\mathscr{D}_I^{\text{src}}$ denote the source rollout windows $(\mathscr{O}_t^H, \mathscr{A}_t^H, Y_{t,\text{exec}}^K, U_{t,\text{exec}}^K)$ extracted from DAgger-style source rollouts, where $U_{t,\text{exec}}^K$ contains the actions actually executed along the rollout rather than privileged oracle labels. To match the receding-horizon execution interface, we supervise only the first action of this chunk:

$$\mathscr{L}_I(\psi) = \mathbb{E}_{(\mathscr{O}_t^H, \mathscr{A}_t^H, Y_{t,\text{exec}}^K, U_{t,\text{exec}}^K)\sim\mathscr{D}_I^{\text{src}}}\left[\left\|\Pi_1\Big(I_\psi(\mathscr{O}_t^H, \mathscr{A}_t^H, Y_{t,\text{exec}}^K)\Big) - \Pi_1(U_{t,\text{exec}}^K)\right\|_2^2\right]. \tag{4.2}$$

Here, $\Pi_1(U_{t,\text{exec}}^K)$ is the first action actually executed in the rollout window. It is not necessarily the privileged oracle action. Thus, the IDM is trained to recover the executed action associated with a realized future, rather than to imitate the oracle directly. Because the supervision comes from DAgger-style rollouts, the IDM is trained on both teacher-guided and suboptimal student executions. This makes the same objective applicable to imperfect target rollouts.

Planner learning: actionable rather than observation-regressed futures. The planner should produce futures that are useful to the IDM under the same receding-horizon interface used at deployment. Let $\mathscr{D}_P^{\text{src}}$ denote source training samples $(\mathscr{O}_t^H, \mathscr{A}_t^H, c_t, a_t^\star)$, where $a_t^\star$ is the privileged oracle action for the visited state, obtained by oracle relabeling as described below. Rather than regressing P_ϕ to an oracle future trajectory, we optimize the planner through the current IDM so that the deployed first action matches the privileged oracle action:

$$\mathscr{L}_P(\phi) = \mathbb{E}_{(\mathscr{O}_t^H, \mathscr{A}_t^H, c_t, a_t^\star)\sim\mathscr{D}_P^{\text{src}}}\left[\left\|\Pi_1\Big(I_{\bar{\psi}}(\mathscr{O}_t^H, \mathscr{A}_t^H, P_\phi(\mathscr{O}_t^H, c_t))\Big) - a_t^\star\right\|_2^2\right], \tag{4.3}$$

where $I_{\bar{\psi}}$ denotes the current IDM with gradients stopped. This trains the planner to produce action-grounded futures: futures that need not match an oracle observation trajectory, but that produce oracle-consistent actions when passed through the IDM. After source training, the planner provides a fixed command-to-intent interface, while the IDM provides the execution module that will be adapted in the target domain.

Oracle relabeling on student rollouts. For rollouts generated by the intermediate Planner–IDM student, the oracle label $a_t^\star$ is obtained by state relabeling. At each visited state we save a simulator snapshot; after the rollout, we restore each snapshot and roll the privileged oracle forward for K steps under the same command, producing an oracle action chunk and the corresponding oracle future-observation chunk $(Y_{\text{orac}}^K, U_{\text{orac}}^K)$. The first action of this chunk supplies the relabeled oracle action $a_t^\star$ used in Eq. (4.3). To keep IDM supervision causally matched, trajectory-source batches use the realized pair $(Y_{\text{traj}}^K, U_{\text{traj}}^K)$, while oracle-source batches use the oracle-shadow pair $(Y_{\text{orac}}^K, U_{\text{orac}}^K)$. The oracle-shadow pair is valid inverse-dynamics supervision because it is generated by physically rolling the oracle forward in the shadow simulator, so its future observations and actions form a causally consistent pair under the oracle policy. Full source-training details are provided in Appendix B.2.

Algorithm 1 FADA procedure for few-shot domain adaptation.

Input. Source-domain oracle policy π^{o} trained with privileged observations and task-specific rewards; target domain ξ_{tgt}.

Output. Target-domain deployable policy $(P_\phi, I_{\psi+\Delta\psi})$.

Source pretraining

S1: Rollout the current source policy in the source domain and append visited-state trajectory pairs $(Y^K_{\mathrm{traj}}, U^K_{\mathrm{traj}})$.

S2: Query the oracle π^{o} at visited states to attach oracle-shadow pairs $(Y^K_{\mathrm{orac}}, U^K_{\mathrm{orac}})$ and first-action labels $a^\star_t$.

S3: Update the IDM I_ψ on the selected matched pair (Y^K_I, U^K_I) using Eq. (4.2).

S4: Update the planner P_ϕ through the fixed IDM using oracle first-action labels and Eq. (4.3).

S5: Repeat S1–S4 as the DAgger source-pretraining loop.

Few-shot target adaptation

T1: Collect target rollouts with the pretrained policy (P_ϕ, I_ψ) and extract $\mathscr{W}_{\mathrm{tgt}}$.

T2: Freeze P_ϕ and ψ, then optimize only $\Delta\psi$ with Eq. (4.5).

T3: Deploy $(P_\phi, I_{\psi+\Delta\psi})$ with planner-predicted futures $\hat{Y}^K_t$ and receding-horizon first-action execution.

Architecture. Both modules are transformer-based. The planner is a transformer encoder over history and command tokens that predicts the future chunk as a residual relative to the latest observation. The IDM is an encoder–decoder in which future-state tokens apply full (non-causal) self-attention over the predicted chunk and cross-attend to the encoded history, and an action head decodes the K-step action chunk in parallel. Architectural details are provided in Appendix B.1.

4.2 Few-Shot Target Adaptation

At deployment, FADA adapts the IDM using the robot's own target-domain rollouts. Given $\mathscr{D}_{\mathrm{tgt}}$, we extract supervision windows

$$\mathscr{W}_{\mathrm{tgt}} = \left\{\left(\mathscr{O}^H_t, \mathscr{A}^H_t, Y^K_{t,\mathrm{exec}}, U^K_{t,\mathrm{exec}}\right)\right\}, \tag{4.4}$$

where $Y^K_{t,\mathrm{exec}}$ and $U^K_{t,\mathrm{exec}}$ are the future proprioceptive sequence and executed action sequence from the same target rollout segment. We denote the target-domain update by $\Delta\psi$, which is implemented as LoRA adapters on the IDM. During adaptation, the planner parameters ϕ and the pretrained IDM weights ψ are frozen, and only $\Delta\psi$ is optimized:

$$\ell_{\mathrm{adapt}}(\Delta\psi) = \mathbb{E}_{(\mathscr{O}^H_t, \mathscr{A}^H_t, Y^K_{t,\mathrm{exec}}, U^K_{t,\mathrm{exec}})\sim\mathscr{W}_{\mathrm{tgt}}}\left[\left\|\Pi_1\Big(I_{\psi+\Delta\psi}(\mathscr{O}^H_t, \mathscr{A}^H_t, Y^K_{t,\mathrm{exec}})\Big) - \Pi_1(U^K_{t,\mathrm{exec}})\right\|^2_2\right]. \tag{4.5}$$

The adapted policy is therefore

$$\pi^{\mathrm{ft}} = (P_\phi, I_{\psi+\Delta\psi}). \tag{4.6}$$

At execution, planner-predicted futures $\hat{Y}^K_t$ are passed to the adapted IDM through the same receding-horizon interface in Eq. (4.1). Thus, FADA changes only how future task intents are realized under target dynamics, while leaving the planner, command interface, and runtime inputs unchanged. The complete source-pretraining and target-adaptation procedure is summarized in Algorithm 1, and full implementation details are provided in Appendix B.

5 Experiments

Our experiments evaluate **FADA** as a few-shot adaptation framework for humanoids by addressing the following questions: **(1)** whether few-shot IDM finetuning improves real-world deployment

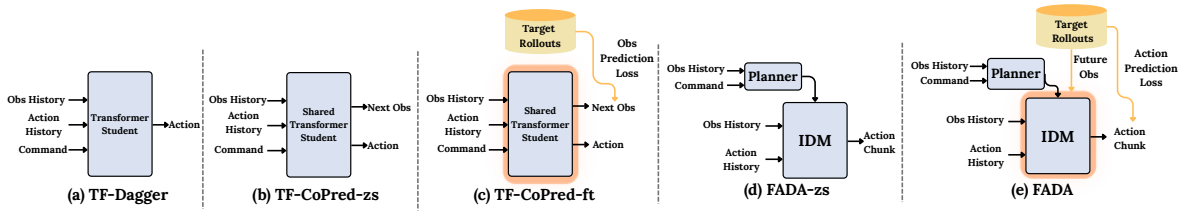

Figure 5: Baseline interfaces. We compare FADA with source-trained transformer DAgger, zero-shot co-prediction, and target-domain co-prediction finetuning. The comparison isolates whether target rollouts are most useful when they update the execution module rather than a monolithic student or a future-prediction objective.

where zero-shot transfer is unreliable, **(2)** whether the framework improves transfer across embodiments, tasks, and controlled sim-to-sim dynamics shifts relative to in-context and future-prediction adaptation baselines, **(3)** how the prediction horizon K affects final performance, and which predicted future steps the IDM relies on, and **(4)** how the planner and IDM training objectives affect whether the zero-shot policy remains deployable enough to collect target rollouts. We further validate the design choices behind this recipe through adaptation-design ablations and a controlled analysis that directly tests the planner–IDM role separation underlying FADA.

Tasks and Systems. We evaluate FADA on two humanoid platforms, Unitree G1 (29 DoF) and Booster T1 (23 DoF), across sim-to-sim (MuJoCo Todorov et al. [2012]) and sim-to-real settings. The task suite covers locomotion and whole-body tracking under dynamics mismatch induced by payload and terrain shifts. All student policies use history length $H = 30$ and prediction horizon $K = 6$ and are trained in IsaacSim Mittal et al. [2023] with domain randomization. Task definitions, deployment conditions, and source-training details are provided in Appendices A.1 and B.2. For target adaptation, locomotion tasks use approximately two minutes of target rollouts at 50 Hz ($\approx$ 6000 control steps), and whole-body tracking tasks use six repetitions of the $\approx$ 20 s reference motion, yielding the same budget; the sim-to-sim adaptation budget is matched to this hardware budget. Unless an ablation states otherwise, sim-to-real results average five hardware trials and main sim-to-sim results average twenty trials.

Baselines and metrics. We compare against three teacher–student baselines distilled from the same privileged oracle as FADA, with their interfaces visualized in Figure 5. *TF–DAgger* is a transformer teacher–student framework Kumar et al. [2021a] that distills a privileged teacher into a history-conditioned policy without target-domain updates. Motivated by recent world-model and world-action-model approaches that learn dynamics-aware representations through future-state or state-action prediction Nahrendra et al. [2023], Long et al. [2023], Lyu et al. [2025], *TF–CoPred-zs* uses a shared transformer backbone to jointly predict the next observation and action without explicit separation between planning and inverse dynamics. *TF–CoPred-ft* finetunes the shared co-prediction backbone on target rollouts using only a future-observation prediction loss, testing whether end-to-end predictive adaptation can improve deployment without adapting a dedicated action-generation module. We additionally denote by *FADA-zs* the pre-adaptation Planner–IDM student (P_ϕ, I_ψ), i.e., FADA evaluated zero-shot before any target-domain update. Full baseline implementations are provided in Appendix B.4.

We report success rate for completion tasks, normalized linear-velocity tracking error $\bar{E}_v$ for locomotion, and normalized mean per-joint position error $\bar{E}_{\text{mpjpe}}$ for whole-body tracking.

5.1 Sim-to-Real Performance after Few-Shot Adaptation

To address **Q1**, we evaluate FADA on five hardware tasks spanning locomotion and whole-body tracking; Table 1 reports results and target-domain shifts, with full task definitions in Appendix A.1, and Figure 6 shows representative rollouts. For each task, we collect target rollouts with the pre-

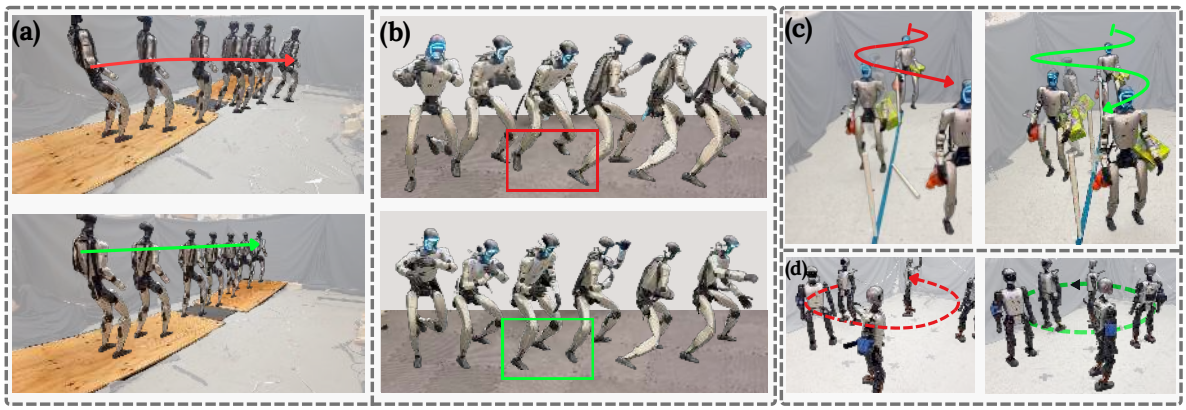

Figure 6: Qualitative sim-to-real deployment. Zero-shot and IDM-adapted rollouts for (a) G1 Slope Traversal, (b) G1 Kungfu + Soft Terrain, (c) G1 Loco. + Payload (grocery carrying through poles), and (d) T1 Loco. + Payload. Adaptation improves execution-critical behavior, including foot placement, posture recovery, and payload compensation.

adaptation policy FADA-zs, freeze the planner, finetune only the IDM, and re-evaluate over 5 hardware trials. We compare against FADA-zs and the transformer teacher–student baseline TF–DAgger.

Table 1: Comparison of normalized sim-to-real task performance across methods. For normalized tracking metrics, TF–DAgger corresponds to 1.0 and lower is better. Success is defined as completing the out-and-back path without leaving the ramp (Slope Traversal) and pulling the basket across the finish line (Basket Pulling); full task definitions are given in Appendix A.1.

Method	G1 Slope Traversal two 15° ramps, ≈ 0.8 m wide		G1 Loco. + Payload ≈ 3 kg asymmetric grocery payload		G1 Kungfu + Soft Terrain deformable soft mats		T1 Loco. + Payload 1 kg asymmetric arm payload		T1 Basket Pulling ≈ 6 kg loaded laundry basket	
	IDM loss (×1e−3) ↓	Success ↑	IDM loss (×1e−3) ↓	$\bar{E}_v$ ↓	IDM loss (×1e−3) ↓	$\bar{E}_{\text{mpjpe}}$ ↓	IDM loss (×1e−3) ↓	$\bar{E}_v$ ↓	IDM loss (×1e−3) ↓	Success ↑
TF–DAgger	–	0%	–	$1.000_{\pm 0.203}$	–	$1.00_{\pm 0.091}$	–	$1.000_{\pm 0.132}$	–	0%
FADA-zs	$2.391_{\pm 1.877}$	20%	$2.195_{\pm 0.465}$	$1.289_{\pm 0.711}$	$5.247_{\pm 0.166}$	$1.18_{\pm 0.103}$	$1.068_{\pm 0.235}$	$1.043_{\pm 0.025}$	$0.873_{\pm 0.171}$	20%
FADA	$2.190_{\pm 1.416}$	80%	$1.848_{\pm 0.282}$	$0.797_{\pm 0.018}$	$4.460_{\pm 0.141}$	$0.86_{\pm 0.084}$	$0.954_{\pm 0.197}$	$0.866_{\pm 0.040}$	$0.650_{\pm 0.0923}$	100%

As shown in Table 1, few-shot IDM adaptation improves all five hardware tasks. On success-rate tasks, FADA increases average success from 20% zero-shot to **90**%, while TF–DAgger fails both tasks. On normalized-error tasks, FADA reduces error by **27.4**% on average relative to FADA-zs and 15.9% relative to TF–DAgger. The qualitative rollouts in Figure 6 show that these gains correspond to better execution during slope contacts, payload-induced posture shifts, and soft-terrain tracking. These performance gains are accompanied by a consistent decrease in IDM loss across all five tasks, indicating that adaptation improves the IDM's alignment with target-domain dynamics. We compute this loss using the same first-action inverse-dynamics objective as in Eq. (4.2): for each policy, the loss is evaluated on rollout windows collected by that policy in the corresponding target domain. Thus, lower IDM loss reflects better target-domain action prediction and explains the improved hardware performance. Notably, on G1 Kungfu + Soft Terrain, FADA improves whole-body tracking despite the deformable mat being outside the source-training distribution. The zero-shot failures also separate into two recurring modes: under terrain or contact mismatch the policy still initiates the correct skill but accumulates foot-placement and posture errors over time, while under payload or dragging forces the planned motion remains reasonable but the action map can no longer realize the intended body motion. IDM finetuning addresses both modes by recalibrating the plan-to-action map from target rollouts, and broader qualitative evidence is reported in Appendix D. These results show that ordinary target-domain observation-action rollouts are sufficient to improve IDM action prediction under deployment dynamics, without target rewards, privileged labels, simulator recalibration, or

Table 2: Sim-to-sim transfer across embodiments and tasks. For each task and method we report normalized task error (lower is better) under two evaluation conditions: the source IsaacSim training condition ($\text{IsaacSim}_{\text{src}}$) and a held-out MuJoCo simulator (MuJoCo). Pre-adaptation rows use only source training data; finetuned rows update with the same target-domain rollout budget on a few-shot deployment trajectory.

Task	G1 Slope Traversal ↓		G1 Kungfu ↓		T1 Loco. + Payload ↓		T1 Slope Traversal ↓		T1 Falcon ↓	
Method	$\text{IsaacSim}_{\text{src}}$	MuJoCo	$\text{IsaacSim}_{\text{src}}$	MuJoCo	$\text{IsaacSim}_{\text{src}}$	MuJoCo	$\text{IsaacSim}_{\text{src}}$	MuJoCo	$\text{IsaacSim}_{\text{src}}$	MuJoCo
TF–DAgger	$1.000_{\pm 0.390}$	$1.000_{\pm 0.439}$	$1.000_{\pm 0.049}$	$1.000_{\pm 0.052}$	$1.000_{\pm 0.221}$	$1.000_{\pm 0.143}$	$1.000_{\pm 0.261}$	$1.000_{\pm 0.252}$	$1.000_{\pm 0.259}$	$1.000_{\pm 0.086}$
TF–CoPred-zs	$1.036_{\pm 0.383}$	$0.973_{\pm 0.387}$	$1.024_{\pm 0.073}$	$1.031_{\pm 0.068}$	$0.991_{\pm 0.219}$	$1.029_{\pm 0.142}$	$1.026_{\pm 0.287}$	$1.080_{\pm 0.224}$	$0.974_{\pm 0.251}$	$0.943_{\pm 0.133}$
TF–CoPred-ft	–	$1.611_{\pm 0.488}$	–	$1.421_{\pm 0.198}$	–	$1.737_{\pm 0.387}$	–	$1.193_{\pm 0.246}$	–	$1.054_{\pm 0.162}$
FADA-zs	$1.038_{\pm 0.386}$	$0.946_{\pm 0.358}$	$0.988_{\pm 0.050}$	$0.961_{\pm 0.055}$	$1.064_{\pm 0.208}$	$1.042_{\pm 0.147}$	$1.019_{\pm 0.243}$	$1.034_{\pm 0.334}$	$0.994_{\pm 0.316}$	$0.880_{\pm 0.094}$
FADA	–	$\mathbf{0.800}_{\pm 0.236}$	–	$\mathbf{0.714}_{\pm 0.048}$	–	$\mathbf{0.885}_{\pm 0.135}$	–	$\mathbf{0.914}_{\pm 0.193}$	–	$\mathbf{0.347}_{\pm 0.043}$

full-policy finetuning.

5.2 Sim-to-Sim Transfer across Embodiments and Tasks

To address **Q2**, we use IsaacSim-to-MuJoCo transfer as a controlled testbed for studying how target-domain adaptation should be applied. We evaluate five tasks across G1 and T1, with all adapted methods using the same few-shot MuJoCo rollout budget and only proprioceptive observation-action trajectories. Two patterns emerge from Table 2. First, in the *source* $\text{IsaacSim}_{\text{src}}$ columns, all zero-shot methods remain close to the *TF–DAgger* reference, indicating that the MuJoCo gains are not due to differences in source-domain performance. Second, under held-out *MuJoCo* dynamics, the advantage appears when target supervision updates the IDM. Averaged across all five tasks, FADA reduces normalized error by **24.7**% over FADA-zs and **26.8**% over *TF–DAgger*. The largest gain appears on T1 Falcon [Zhang et al., 2025a], a force-adaptive locomotion task in which the robot must track commanded velocities while compensating for a persistent external pulling force. Because performance in this task depends strongly on accurately compensating for external forces, it is especially sensitive to dynamics mismatch and therefore benefits from IDM adaptation. In contrast, *TF–CoPred-ft* worsens relative to *TF–CoPred-zs*, suggesting that improving future-observation prediction alone does not guarantee better deployment performance when the action-generation map remains mismatched. These results support adapting the IDM with target rollouts as a way to improve transfer without changing the planner or policy interface. These gains are also not an artifact of cross-simulator implementation differences: under same-simulator (IsaacSim-to-IsaacSim) held-out payload and terrain shifts, evaluated over 1024 parallel environments, FADA still consistently improves over zero-shot transfer (Appendix C.2).

5.3 Influence of Prediction Horizon on Performance

The prediction horizon K couples the planner and IDM: the planner predicts K future proprioceptive observations, and the IDM attends to this window to output an action chunk whose first action is deployed. To address **Q3**, we study both how much future context is needed and how the IDM uses that context. We evaluate this through: (i) a sweep over $K \in \{1, 6, 10, 15\}$ on post-adaptation IsaacSim-to-MuJoCo transfer for two representative tasks; (ii) a *visible-prefix* study on a trained K=6 model, where only the first k predicted observations are visible and the remaining $6 - k$ are masked, and (iii) a *leave-one-out* study, where each predicted step is masked individually to measure its marginal importance.

K	G1 WBT ($\bar{E}_{\text{mpjpe}} \downarrow$)	T1 Locomotion ($\bar{E}_v \downarrow$)
1	1.000±0.136	1.000±0.129
6	**0.830±0.107**	**0.813±0.122**
10	0.852±0.101	0.931±0.121
15	0.858±0.135	0.926±0.112

Table 3: Horizon K ablation (post-adaptation). Normalized error (lower is better) for G1 tracking ($\bar{E}_{\text{mpjpe}}$) and T1 Loco. + Payload ($\bar{E}_v$) after few-shot adaptation.

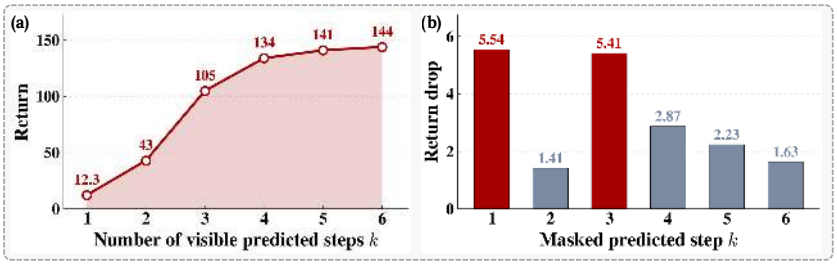

Figure 7: Attribution over predicted future steps for K=6. (a) *Visible-prefix*: only the first k predicted observations are given to the IDM. (b) *Leave-one-out*: one predicted step is masked and the return drop is measured.

Table 3 shows that K=1 is myopic: moving to K=6 reduces normalized error by about 18% on both tasks, while longer horizons provide no further gain. This suggests that useful information is concentrated in a compact future window. Figure 7 further explains this behavior: The *visible-prefix analysis* shows that most of the gain appears once the IDM sees the first few future tokens, while the *leave-one-out* analysis identifies the **first** and **third** predicted steps as the most influential. Importantly, although the IDM is supervised only on the first executed action (Section 4.1), its decoder uses full attention over the predicted future tokens, allowing later predictions to shape the deployed action. This suggests that the IDM is not learning a purely one-step inverse model, but a higher-order plan-to-action map that uses short- and medium-horizon predictions to infer local motion trend and execution context. Consistent with this, replacing the attention-based modules with MLP counterparts leaves source-domain performance nearly unchanged but removes most of the post-adaptation benefit (Appendix C.1), indicating that attention over history and future tokens is what makes the IDM adaptable.

5.4 Effect of Planner and IDM Training Objectives on Zero-Shot Transfer

Few-shot adaptation requires the zero-shot policy to remain deployable long enough to collect target rollouts. We therefore ablate on two key design choices in our training losses from Section 4, Eqs. (4.2) and (4.3) that affect zero-shot transfer before any target update is applied. (A) trains the planner through a frozen IDM using action-prediction loss, rather than directly regressing the planner to oracle future observations (B) supervises the IDM only on the first executed action, rather than on the full predicted action window. We evaluate FADA and the three ablations on zero-shot IsaacSim-to-MuJoCo transfer for G1 whole-body tracking. As shown in Figure 8, removing (B) causes the larger drop, reducing success from **10/10** to **4/10**, indicating that full-window action supervision can hurt receding-horizon deployment when only the first action is executed. Removing (A) reduces success to 7/10, suggesting that observation-regressed planner outputs can be plausible but not necessarily actionable by the IDM. Removing both choices compounds the failure to **1/10**. Together, these results show that (A) keeps planner predictions compatible with the IDM, while (B) aligns IDM training with the action actually applied by the robot.

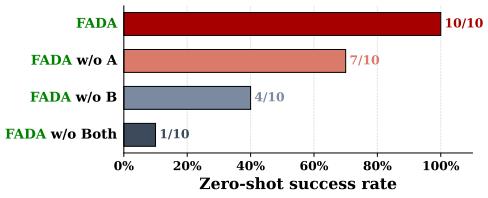

Figure 8: Zero-shot transfer to MuJoCo on G1 whole-body tracking ($n = 10$). Our loss formulation in Section 4 has two design choices: (A) training the planner through the stop-gradient IDM via action-prediction loss (Eq. (4.3)); (B) supervising the IDM only on the executed first action (Eq. (4.2)).

5.5 Adaptation Design Ablations

We next ablate the two design choices that define FADA's adaptation recipe: updating the IDM through low-rank adapters, and the size of the target rollout budget. Both ablations are run on MuJoCo sim-to-sim targets with 20 randomized-command trials per task, and results are normalized column-wise by the first row.

Table 4: LoRA vs. full IDM finetuning. We report $\bar{E}_v \downarrow$ on three MuJoCo target tasks. LoRA provides the most stable few-shot adaptation, while full IDM finetuning can overfit and degrade the policy.

Method	G1 Slope Traversal	T1 Loco. + Payload	T1 Slope Traversal
FADA-zs	$1.0000_{\pm 0.3788}$	$1.0000_{\pm 0.1413}$	$1.0000_{\pm 0.3230}$
Full IDM FT	$1.1000_{\pm 0.4484}$	$1.1163_{\pm 0.3342}$	$1.2918_{\pm 0.2819}$
LoRA IDM (ours)	$\mathbf{0.8463}_{\pm 0.2493}$	$\mathbf{0.8489}_{\pm 0.1293}$	$\mathbf{0.8846}_{\pm 0.1870}$

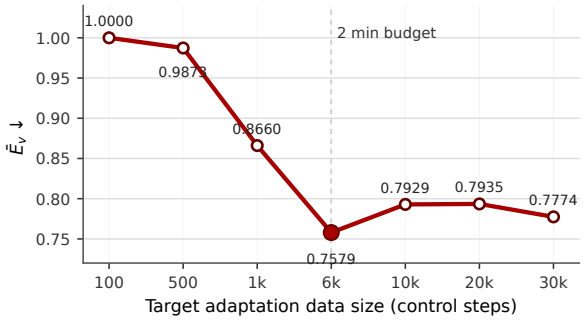

Figure 9: Target data-size ablation. We report $\bar{E}_v \downarrow$ on T1 Loco. + Payload, normalized by the 100-step setting. The 6000-step budget used in the main experiments reaches the performance plateau, and larger budgets do not provide consistent gains.

LoRA vs. full IDM finetuning. Table 4 compares the default LoRA adaptation against directly finetuning all IDM parameters. LoRA is consistently the strongest variant across all three target tasks. Full IDM finetuning, in contrast, often degrades performance relative to the zero-shot policy, indicating that updating the entire action-generation module overfits to the limited target rollout budget. This is consistent with FADA's premise that the few-shot regime requires a constrained execution-side update rather than a full retraining of the IDM.

Target rollout budget. Figure 9 varies the amount of target rollout data on T1 Loco. + Payload. Performance improves quickly from very small budgets to the 6000-step setting used in the main experiments, then saturates. This supports the few-shot framing of FADA: roughly two minutes of target rollouts capture most of the adaptation benefit, and collecting substantially more data brings only marginal additional gain.

5.6 Controlled Analysis of Planner–IDM Role Separation

Finally, we use a fixed-base arm task as a controlled diagnostic for the central hypothesis of Section 4: that dynamics shifts change how an intent must be executed, not the intent itself. A 7-DoF arm tracks a sequence of Cartesian end-effector targets while the only domain shift is a wrist payload $m \in \{0, 2.5, 5.0\}$ kg (Figure 10). The source model is trained with payload randomization over $m \in [0, 1.5]$ kg, so the heavier payloads test extrapolation beyond the source distribution. This setting separates kinematics from dynamics: the joint configuration needed to reach a target is unchanged by the payload, whereas the action required to realize and hold that configuration changes with mass. The payload-induced gravity torque is configuration-dependent, so successful adaptation requires more than a constant action offset.

We evaluate three diagnostics before and after few-shot LoRA finetuning of the IDM, with the planner frozen: end-effector tracking error, planner prediction RMSE, and an IDM consistency gap. The consistency gap compares the IDM action produced using the planner-predicted next observation with the IDM action produced using the actual next observation from the same rollout. A smaller gap indicates that the executed trajectory is closer to the planner-predicted trajectory, making the deployed and teacher-forced IDM inputs more consistent.

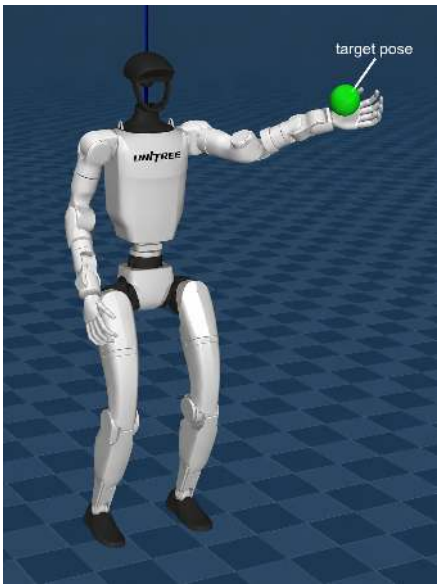

Figure 10: Arm-tracking task. A fixed-base arm tracks end-effector targets under wrist payloads.

Planner invariance. Figure 11 shows that the planner predicts nearly identical joint configurations for the same end-effector targets across all payloads. The mean per-joint planner RMSE changes by only **7%** from m=0 to m=5 kg (0.049 vs. 0.052 rad), despite the heavier payload being outside the training range. This supports the intended role of the planner: it captures a payload-invariant kinematic map rather than compensating for payload-dependent dynamics.

IDM adaptation. Figure 12 shows that finetuning only the IDM improves both tracking and interface consistency. Across payloads, the consistency gap decreases by **≈54%** on average, from 0.079 to 0.037 rad, while end-effector tracking error decreases by **≈24%**. The planner RMSE also decreases by **≈39%**, even though the planner weights are frozen. This does not indicate that the planner changed; rather, the adapted IDM drives the arm toward states that better match the planner's original predictions. The improvement is not merely a payload-specific action bias, because the wrist load induces configuration-dependent gravity torques, so the required correction varies with the target, posture, and transient motion between targets. The reduction in both tracking error and consistency gap therefore suggests that IDM finetuning learns a structured, plan-conditioned inverse-dynamics correction using the predicted future and recent execution history. While this separation is cleanest in the fixed-base setting, it is consistent with the humanoid results in Sections 5.1 and 5.2, where adaptation gains concentrate in execution-critical behaviors such as foot placement, posture recovery, and payload compensation. Overall, this controlled task supports the role separation used by FADA: the planner retains the command-to-kinematics map, while few-shot IDM adaptation recalibrates the dynamics-dependent plan-to-action map.

6 Conclusion

We presented FADA, a few-shot domain adaptation framework for humanoid control that aligns target-domain dynamics using only deployment rollouts. FADA factorizes a deployable policy into a planner that specifies future task intent and an IDM that realizes this intent as actions, then adapts only the IDM to the target domain. In both controlled sim-to-sim transfer and real-world deployment on Unitree G1 and Booster T1, FADA improves task performance under dynamics shifts and enables real humanoid robots to execute diverse high-precision whole-body tasks. These results suggest that many humanoid transfer failures are not failures of task intent, but failures of realizing that intent under new physics. By separating intent generation from execution, FADA offers a practical path toward scalable post-deployment adaptation for high-precision humanoid control.

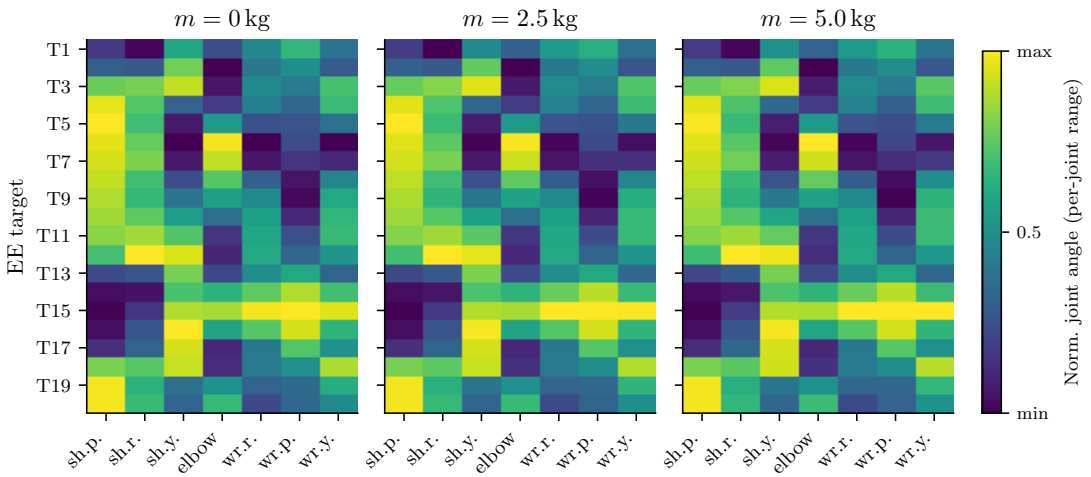

Figure 11: Planner IK map across payload conditions before finetuning. Each cell shows the mean joint angle predicted by the planner for a target and joint, with colors normalized per joint across all payloads. The near-identical patterns show that the planner maps the same end-effector target to the same joint configuration regardless of payload.

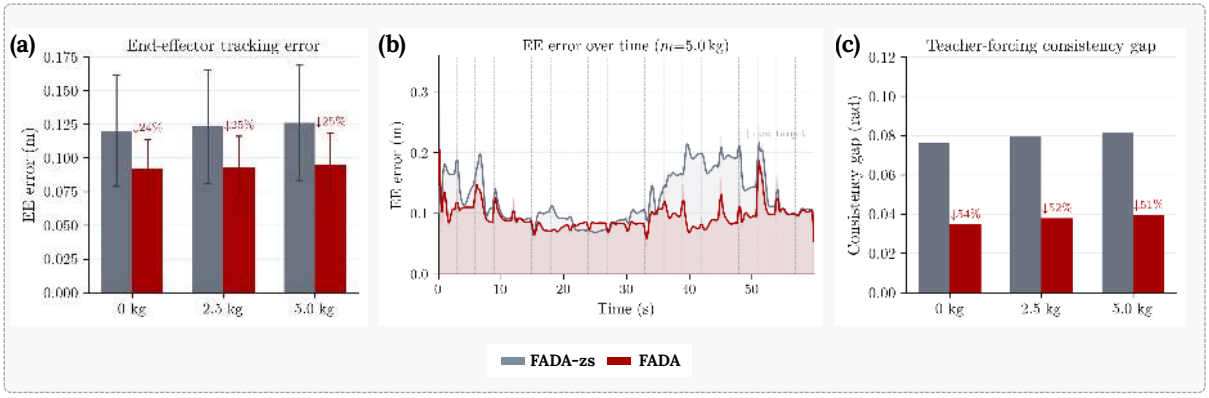

Figure 12: Fixed-base arm tracking under payload variation. **(a)** End-effector tracking error before and after IDM finetuning. **(b)** Per-timestep EE error at m=5.0 kg; vertical dashes mark command switches every 3 s. **(c)** IDM consistency gap between actions computed from planner-predicted and actual next observations. Annotated percentages indicate reductions after adaptation.

7 Limitations

While these results are encouraging, the current formulation also has several important limitations.

Dependence on non-trivial zero-shot performance. FADA requires the source-trained policy to collect target rollouts with useful observation-action correspondences. Catastrophic zero-shot failures may therefore require safety mechanisms, recovery controllers, or assisted data collection.

Task-dependent execution module. Although the IDM is intended to capture execution dynamics, it is trained within a particular task distribution and may still encode task-specific structure. A more task-agnostic execution model that transfers across tasks could further reduce adaptation cost and broaden the applicability of the framework.

Proprioception-only adaptation. The current instantiation adapts from proprioceptive observations and executed actions without explicitly representing external factors such as terrain geometry

or payload distribution. While this avoids environment reconstruction, extending the same principle to richer perceptual inputs or learned world-model representations could improve adaptation when the mismatch is primarily exteroceptive.

Acknowledgements

We thank Yuxiang Yang for his insightful feedback and suggestions.

References

Nico Bohlinger and Jan Peters. Multi-embodiment locomotion at scale with extreme embodiment randomization. *arXiv preprint arXiv:2509.02815*, 2025.

Woohyun Cha, Junhyeok Cha, Jaeyong Shin, Donghyeon Kim, and Jaeheung Park. Sim-to-Real of Humanoid Locomotion Policies via Joint Torque Space Perturbation Injection, April 2025. URL http://arxiv.org/abs/2504.06585. arXiv:2504.06585 [cs.RO] version: 1.

Leixin Chang, Yuxuan Nai, Hua Chen, and Liangjing Yang. Beyond robustness: Learning unknown dynamic load adaptation for quadruped locomotion on rough terrain. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025. doi: 10.1109/ICRA55743.2025.11128639. URL https://arxiv.org/abs/2507.07825.

Sirui Chen, Keenon Werling, Albert Wu, and C. Karen Liu. Real-time Model Predictive Control and System Identification Using Differentiable Physics Simulation, 2022. URL https://arxiv.org/abs/2202.09834.

Jin Cheng, Dongho Kang, Gabriele Fadini, Guanya Shi, and Stelian Coros. RAMBO: RL-augmented Model-based Optimal Control for Whole-body Loco-manipulation, 2025. URL https://arxiv.org/abs/2504.06662.

Xuxin Cheng, Yandong Ji, Junming Chen, Ruihan Yang, Ge Yang, and Xiaolong Wang. Expressive whole-body control for humanoid robots, 2024.

Wenhao Cui, Shengtao Li, Huaxing Huang, Bangyu Qin, Tianchu Zhang, Jinchao Han, Liang Zheng, Ziyang Tang, Chenxu Hu, Ning Yan, Jiahao Chen, and Zheyuan Jiang. Adapting humanoid locomotion over challenging terrain via two-phase training. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 3439–3453. PMLR, 2025. URL https://proceedings.mlr.press/v270/cui25a.html.

Longchao Da, Justin Turnau, Thirulogasankar Pranav Kutralingam, Alvaro Velasquez, Paulo Shakarian, and Hua Wei. A Survey of Sim-to-Real Methods in RL: Progress, Prospects and Challenges with Foundation Models, 2025. URL https://arxiv.org/abs/2502.13187.

Zipeng Fu, Qingqing Zhao, Qi Wu, Gordon Wetzstein, and Chelsea Finn. Humanplus: Humanoid shadowing and imitation from humans. *arXiv preprint arXiv:2406.10454*, 2024.

Xinyang Gu, Yen-Jen Wang, and Jianyu Chen. Humanoid-gym: Reinforcement learning for humanoid robot with zero-shot sim2real transfer, 2024.

Tairan He, Zhengyi Luo, Xialin He, Wenli Xiao, Chong Zhang, Weinan Zhang, Kris Kitani, Changliu Liu, and Guanya Shi. Omnih2o: Universal and dexterous human-to-humanoid whole-body teleoperation and learning. *arXiv preprint arXiv:2406.08858*, 2024a.

Tairan He, Wenli Xiao, Toru Lin, Zhengyi Luo, Zhenjia Xu, Zhenyu Jiang, Jan Kautz, Changliu Liu, Guanya Shi, Xiaolong Wang, Linxi Fan, and Yuke Zhu. Hover: Versatile neural whole-body controller for humanoid robots. *arXiv preprint arXiv:2410.21229*, 2024b.

Tairan He, Jiawei Gao, Wenli Xiao, Yuanhang Zhang, Zi Wang, Jiashun Wang, Zhengyi Luo, Guanqi He, Nikhil Sobanbab, Chaoyi Pan, Zeji Yi, Guannan Qu, Kris Kitani, Jessica Hodgins, Linxi "Jim" Fan, Yuke Zhu, Changliu Liu, and Guanya Shi. ASAP: Aligning Simulation and Real-World Physics for Learning Agile Humanoid Whole-Body Skills, April 2025. URL http://arxiv.org/abs/2502.01143. arXiv:2502.01143 [cs].

Kaizhe Hu, Haochen Shi, Yao He, Weizhuo Wang, C. Karen Liu, and Shuran Song. Robot trains robot: Automatic real-world policy adaptation and learning for humanoids. *arXiv preprint arXiv:2508.12252*, 2025.

Haodong Huang, Shilong Sun, Yuanpeng Wang, Chiyao Li, Hailin Huang, and Wenfu Xu. BarlowWalk: Self-supervised Representation Learning for Legged Robot Terrain-adaptive Locomotion, 2025. URL https://arxiv.org/abs/2508.00939.

Weidong Huang, Zhehan Li, Hangxin Liu, Biao Hou, Yao Su, and Jingwen Zhang. Towards Bridging the Gap between Large-Scale Pretraining and Efficient Finetuning for Humanoid Control, 2026. URL https://arxiv.org/abs/2601.21363.

Yunfan Jiang, Chen Wang, Ruohan Zhang, Jiajun Wu, and Li Fei-Fei. TRANSIC: Sim-to-Real Policy Transfer by Learning from Online Correction, 2024. URL https://arxiv.org/abs/2405.10315.

Joshua Jones, Oier Mees, Carmelo Sferrazza, Kyle Stachowicz, Pieter Abbeel, and Sergey Levine. Beyond Sight: Finetuning Generalist Robot Policies with Heterogeneous Sensors via Language Grounding, 2025. URL https://arxiv.org/abs/2501.04693.

Lokesh Krishna, Sheng Cheng, Junheng Li, Naira Hovakimyan, and Quan Nguyen. DiffCoTune: Differentiable Co-Tuning for Cross-domain Robot Control, June 2025. URL http://arxiv.org/abs/2505.24068. arXiv:2505.24068 [cs].

Ashish Kumar, Zipeng Fu, Deepak Pathak, and Jitendra Malik. RMA: Rapid Motor Adaptation for Legged Robots, July 2021a. URL http://arxiv.org/abs/2107.04034. arXiv:2107.04034.

Ashish Kumar, Zhongyu Li, Jun Zeng, Deepak Pathak, Koushil Sreenath, and Jitendra Malik. Adapting Rapid Motor Adaptation for Bipedal Robots, September 2022. URL http://arxiv.org/abs/2205.15299. arXiv:2205.15299.

Visak Kumar, Sehoon Ha, and C. Karen Liu. Error-Aware Policy Learning: Zero-Shot Generalization in Partially Observable Dynamic Environments, March 2021b. URL http://arxiv.org/abs/2103.07732. arXiv:2103.07732.

Easop Lee, Samuel A. Moore, and Boyuan Chen. Sym2Real: Symbolic Dynamics with Residual Learning for Data-Efficient Adaptive Control, 2025. URL https://arxiv.org/abs/2509.15412.

Kun Lei, Zhengmao He, Chenhao Lu, Kaizhe Hu, Yang Gao, and Huazhe Xu. Uni-O4: Unifying Online and Offline Deep Reinforcement Learning with Multi-Step On-Policy Optimization, March 2024. URL http://arxiv.org/abs/2311.03351. arXiv:2311.03351 [cs].

Yu Lei, Minghuan Liu, Abhiram Maddukuri, Zhenyu Jiang, and Yuke Zhu. A Mechanistic Analysis of Sim-and-Real Co-Training in Generative Robot Policies, April 2026. URL http://arxiv.org/abs/2604.13645. arXiv:2604.13645 [cs].

Jacob Levy, Tyler Westenbroek, Kevin Huang, Fernando Palafox, Patrick Yin, Shayegan Omidshafiei, Dong-Ki Kim, Abhishek Gupta, and David Fridovich-Keil. Simulation distillation: Pretraining world models in simulation for rapid real-world adaptation, 2026. Robotics: Science and Systems 2026.

Chenhao Li, Andreas Krause, and Marco Hutter. Robotic World Model: A Neural Network Simulator for Robust Policy Optimization in Robotics, December 2025. URL http://arxiv.org/abs/2501.10100. arXiv:2501.10100 [cs].

Chenhao Li, Andreas Krause, and Marco Hutter. Uncertainty-Aware Robotic World Model Makes Offline Model-Based Reinforcement Learning Work on Real Robots, January 2026. URL http://arxiv.org/abs/2504.16680. arXiv:2504.16680 [cs].

Min Liu, Deepak Pathak, and Ananye Agarwal. LocoFormer: Generalist Locomotion via Long-context Adaptation, September 2025. URL http://arxiv.org/abs/2509.23745. arXiv:2509.23745 [cs].

Junfeng Long, Zirui Wang, Quanyi Li, Jiawei Gao, Liu Cao, and Jiangmiao Pang. Hybrid Internal Model: Learning Agile Legged Locomotion with Simulated Robot Response, 2023. URL https://arxiv.org/abs/2312.11460.

Jiangran Lyu, Ziming Li, Xuesong Shi, Chaoyi Xu, Yizhou Wang, and He Wang. DyWA: Dynamics-adaptive World Action Model for Generalizable Non-prehensile Manipulation, July 2025. URL http://arxiv.org/abs/2503.16806. arXiv:2503.16806 [cs].

Abhiram Maddukuri, Zhenyu Jiang, Lawrence Yunliang Chen, Soroush Nasiriany, Yuqi Xie, Yu Fang, Wenqi Huang, Zu Wang, Zhenjia Xu, Nikita Chernyadev, Scott Reed, Ken Goldberg, Ajay Mandlekar, Linxi Fan, and Yuke Zhu. Sim-and-Real Co-Training: A Simple Recipe for Vision-Based Robotic Manipulation, 2025. URL https://arxiv.org/abs/2503.24361. Version Number: 2.

Mayank Mittal, Calvin Yu, Qinxi Yu, Jingzhou Liu, Nikita Rudin, David Hoeller, Jia Lin Yuan, Ritvik Singh, Yunrong Guo, Hammad Mazhar, Ajay Mandlekar, Buck Babich, Gavriel State, Marco Hutter, and Animesh Garg. Orbit: A unified simulation framework for interactive robot learning environments. *IEEE Robotics and Automation Letters*, 8(6):3740–3747, 2023. doi: 10.1109/LRA.2023.3270034. arXiv:2301.04195.

I. Made Aswin Nahrendra, Byeongho Yu, and Hyun Myung. DreamWaQ: Learning Robust Quadrupedal Locomotion With Implicit Terrain Imagination via Deep Reinforcement Learning, March 2023. URL http://arxiv.org/abs/2301.10602. arXiv:2301.10602.

Allen Z. Ren, Hongkai Dai, Benjamin Burchfiel, and Anirudha Majumdar. AdaptSim: Task-Driven Simulation Adaptation for Sim-to-Real Transfer, 2023. URL https://arxiv.org/abs/2302.04903.

Younggyo Seo, Carmelo Sferrazza, Juyue Chen, Guanya Shi, Rocky Duan, and Pieter Abbeel. Learning Sim-to-Real Humanoid Locomotion in 15 Minutes, December 2025. URL http://arxiv.org/abs/2512.01996. arXiv:2512.01996 [cs.RO].

Jean Pierre Sleiman, He Li, Alphonsus Adu-Bredu, Robin Deits, Arun Kumar, Kevin Bergamin, Mohak Bhardwaj, Scott Biddlestone, Nicola Burger, Matthew A. Estrada, Francesco Iacobelli, Twan Koolen, Alexander Lambert, Erica Lin, M. Eva Mungai, Zach Nobles, Shane Rozen-Levy, Yuyao Shi, Jiashun Wang, Jakob Welner, Fangzhou Yu, Mike Zhang, Alfred Rizzi, Jessica Hodgins, Sylvain Bertrand, Yeuhi Abe, Scott Kuindersma, and Farbod Farshidian. ZEST: Zero-shot Embodied Skill Transfer for Athletic Robot Control, January 2026. URL http://arxiv.org/abs/2602.00401. arXiv:2602.00401 [cs].

Laura Smith, J. Chase Kew, Xue Bin Peng, Sehoon Ha, Jie Tan, and Sergey Levine. Legged Robots that Keep on Learning: Fine-Tuning Locomotion Policies in the Real World, 2021. URL https://arxiv.org/abs/2110.05457.

Nikhil Sobanbabu, Guanqi He, Tairan He, Yuxiang Yang, and Guanya Shi. Sampling-based system identification with active exploration for legged sim2real learning. In Joseph Lim, Shuran Song, and Hae-Won Park, editors, *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 578–598. PMLR, 9 2025. URL https://proceedings.mlr.press/v305/sobanbabu25a.html.

Wandong Sun, Long Chen, Yongbo Su, Baoshi Cao, Yang Liu, and Zongwu Xie. Learning Humanoid Locomotion with World Model Reconstruction, February 2025. URL http://arxiv.org/abs/2502.16230. arXiv:2502.16230 [cs.RO] version: 1.

Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012. doi: 10.1109/IROS.2012.6386109.

Yunshen Wang, Shaohang Zhu, Peiyuan Zhi, Yuhan Li, Jiaxin Li, Yong-Lu Li, Yuchen Xiao, Xingxing Wang, Baoxiong Jia, and Siyuan Huang. OmniXtreme: Breaking the Generality Barrier in High-Dynamic Humanoid Control, February 2026. URL http://arxiv.org/abs/2602.23843. arXiv:2602.23843 [cs].

Weiji Xie, Chenjia Bai, Jiyuan Shi, Junkai Yang, Yunfei Ge, Weinan Zhang, and Xuelong Li. Humanoid Whole-Body Locomotion on Narrow Terrain via Dynamic Balance and Reinforcement Learning, February 2025. URL http://arxiv.org/abs/2502.17219. arXiv:2502.17219.

Jie Xue, Zhiyuan Liang, Haiming Mou, Qingdu Li, and Jianwei Zhang. LSWM: A Long-Short History World Model for Bipedal Locomotion via Reinforcement Learning. *Biomimetics*, 11(1):40, January 2026. ISSN 2313-7673. doi: 10.3390/biomimetics11010040.

Yuanhang Zhang, Yifu Yuan, Prajwal Gurunath, Ishita Gupta, Shayegan Omidshafiei, Ali-akbar Aghamohammadi, Marcell Vazquez-Chanlatte, Liam Pedersen, Tairan He, and Guanya Shi. Falcon: Learning force-adaptive humanoid loco-manipulation. *arXiv preprint arXiv:2505.06776*, 2025a.

Zhikai Zhang, Jun Guo, Chao Chen, Jilong Wang, Chenghuai Lin, Yunrui Lian, Han Xue, Zhenrong Wang, Maoqi Liu, Jiangran Lyu, Huaping Liu, He Wang, and Li Yi. Track Any Motions under Any Disturbances, October 2025b. URL http://arxiv.org/abs/2509.13833. arXiv:2509.13833 [cs].

Siheng Zhao, Yanjie Ze, Yue Wang, C. Karen Liu, Pieter Abbeel, Guanya Shi, and Rocky Duan. ResMimic: From General Motion Tracking to Humanoid Whole-body Loco-Manipulation via Residual Learning, October 2025. URL http://arxiv.org/abs/2510.05070. arXiv:2510.05070 [cs.RO].

Appendix

A Experimental Setup

This appendix provides additional setup details for the experiments in Section 5. We focus on the task definitions, deployment shifts, data budgets, and metrics needed to reproduce the comparisons in the main text. All source policies are trained in IsaacSim, while target evaluation is performed either on hardware or in held-out simulator conditions.

A.1 Tasks and Deployment Conditions

Each experiment is defined by a task $\mathcal{T}$, a source condition $\xi_{\text{src}} \sim \Omega_{\text{src}}$, and a target deployment condition ξ_{tgt}. The task objective remains unchanged across domains, while the transition dynamics may differ because of changes in payload, terrain contact, actuator behavior, latency, or simulator implementation. The goal is to adapt a policy trained under Ω_{src} so that it maintains task performance under ξ_{tgt} using only a small amount of target-domain interaction data.

Hardware deployment tasks. For sim-to-real evaluation, we deploy on Unitree G1 and Booster T1 across locomotion, whole-body tracking, and loco-manipulation settings. The main hardware benchmark contains five tasks.

G1 Slope Traversal. Unitree G1 traverses two 15° slopes on a narrow ramp of width approximately 0.8 m. The robot walks forward, turns 180°, and walks back over a total path length of approximately 20 m. A rollout is successful only if the robot completes the full out-and-back trajectory without stepping off the ramp.

G1 Loco. + Payload. Unitree G1 carries asymmetric grocery loads while following a sinusoidal path through three poles. The left hand carries a snack box, and the right hand carries a bag of oranges. The total load is approximately 3 kg, with the oranges being noticeably heavier and the snack box partially obstructing leg motion. We evaluate this task using normalized velocity-tracking error along the commanded path.

G1 Kungfu + Soft Terrain. G1 tracks a Kungfu reference motion on deformable soft mats. Each hardware motion lasts approximately 15 s. We report normalized mean per-joint position error (MPJPE) and treat the rollout as failed if the robot falls before completing the motion.

T1 Loco. + Payload. Booster T1 tracks a circular path of radius 1 m while carrying an asymmetric 1 kg payload on the right arm. We evaluate this task using normalized velocity-tracking error around the commanded circle.

T1 Basket Pulling. T1 pulls a plastic laundry basket loaded with laundry and weights, with a total mass of approximately 6 kg. The commanded motion is a backward straight-line track of approximately 6 m. A rollout is considered successful if the robot pulls the basket across the finish line.

G1 payload dancing. We include this as an additional qualitative hardware deployment rather than as part of the five-task quantitative benchmark. G1 tracks a dancing motion while carrying a front-mounted bag of approximately 3.2 kg for a 20 s motion. We use it to illustrate stability and motion completion after adaptation.

Sim-to-sim tasks. For sim-to-sim evaluation, policies are trained in IsaacSim and evaluated under held-out MuJoCo target dynamics. Commands are randomized for all sim-to-sim tasks.

G1 Slope Traversal. G1 follows randomized velocity commands on 20° slopes. We evaluate this task using normalized velocity-tracking error.

G1 Kungfu. G1 tracks the Kungfu reference motion under MuJoCo dynamics. The sequence is longer than the hardware deployment sequence, with a total duration of approximately 60 s. We evaluate this task using normalized mean per-joint position error (MPJPE).

T1 Loco. + Payload. T1 follows randomized velocity commands with a 5 kg payload mounted on the torso. We evaluate this task using normalized velocity-tracking error.

T1 Slope Traversal. T1 follows randomized velocity commands on 10° slopes. We evaluate this task using normalized velocity-tracking error.

T1 Falcon. T1 tracks commanded velocities while a constant 30 N downward force is applied to the right hand and no external force is applied to the left hand. We evaluate this force-adaptive locomotion task using normalized velocity-tracking error.

A.2 Observation Space

Table 5 summarizes the observations available to the deployable student and to the privileged oracle. The student observes only quantities measurable at deployment—proprioception, the task command, and a gait or motion reference—while the oracle additionally observes privileged simulator state such as base linear velocity, contact forces, terrain geometry, actuator parameters, and the sampled randomization parameters. The two policy families (whole-body tracking and locomotion/loco-manipulation) share the same observation structure but differ in a few per-term scales, which we note in the table. All policies use a history of H=30 stacked frames and a prediction horizon of K=6, as described in Section 5.

A.3 Domain Randomization

All source policies are trained in IsaacSim under domain randomization over dynamics, actuation, and perturbation parameters. Table 6 lists the sampled ranges. The whole-body tracking and locomotion families use the same randomization scheme with different magnitudes; whole-body motion tracking additionally randomizes the physical properties of the carried object. These ranges define the source condition distribution Ω_{src}; the target conditions ξ_{tgt} evaluated in the main text are fixed deployment conditions that are not assumed to lie within these ranges.

A.4 Metrics and Normalization

We report task-appropriate metrics that separate binary completion from continuous tracking quality. Completion tasks are reported as success rate. Tracking tasks are reported as normalized errors, where lower is better and 1.0 corresponds to the TF–DAgger baseline error for the same task and evaluation condition. For compact notation, we denote the corresponding TF–DAgger baseline error by the superscript TFD in the formulas below.

Velocity tracking. For locomotion tasks, we compute linear and angular velocity RMSE over the evaluation window:

$$E_{\text{lin}} = \sqrt{\frac{1}{T}\sum_{t=1}^{T}\left\| v^{xy}_{t,\text{cmd}} - v^{xy}_{t,\text{base}} \right\|_2^2}, \qquad E_{\text{ang}} = \sqrt{\frac{1}{T}\sum_{t=1}^{T}\left(\omega^{z}_{t,\text{cmd}} - \omega^{z}_{t,\text{base}} \right)^2}. \tag{A.1}$$

Table 5: Observation space. The deployable student observes proprioception, the task command, and a gait/motion reference; the oracle additionally observes the privileged simulator state listed in the bottom block. The listed scale is the multiplicative factor applied to each term before concatenation; the privileged terms are compact state *bundles* that group several quantities (e.g. contact forces, terrain heights, actuator parameters) and are internally normalized to comparable magnitudes within the observation function, so an outer scale of 1.0 passes them through unchanged. Scales are the locomotion-family values; whole-body-tracking (WBT) values are noted in parentheses where they differ.

Symbol	Description	Scale
	Common student observations (all tasks)	
$\boldsymbol{\omega}_t^{\text{base}}$	Base angular velocity	0.25 (WBT 1.0)
$\mathbf{g}_t$	Projected gravity vector	1.0
$\mathbf{q}_t$	Joint positions	1.0
$\dot{\mathbf{q}}_t$	Joint velocities	0.05 (WBT 1.0)
$\mathbf{a}_{t-1}$	Previous action	1.0
	Task-specific student observations	
v_{cmd}^x, v_{cmd}^y, ω_{cmd}^z	Velocity command (locomotion)	1.0
$\sin\Phi$, $\cos\Phi$	Gait phase (locomotion)	1.0
$q_{\text{upper}}^{\text{ref}}$	Upper-body reference pose (loco-manip.)	1.0
h_{cmd}	Base-height command (loco-manip.)	2.0
$\mathbf{m}_t$	Motion command (WBT)	1.0
R_b^{ref}	Reference orientation in base frame (WBT)	1.0
	Privileged observations (oracle only)	
$\mathbf{v}_t^{\text{base}}$	Base linear velocity	2.0 (WBT 1.0)
$\mathbf{c}_t$	Per-body contact forces and binary contacts	1.0
$\mathbf{h}_t$	Local terrain height profile and root clearance	1.0
$\boldsymbol{\theta}_t^{\text{act}}$	Actuator state (PD-gain scales, normalized torques)	1.0
$\boldsymbol{\xi}_t$	Sampled domain-randomization parameters	1.0
$\mathbf{p}_b^{\text{ref}}, R_b^{\text{ref}}$	Reference body poses in base frame (WBT)	1.0
$\mathbf{f}_t^{\text{ee}}$	End-effector applied force (loco-manip.)	0.1

Table 6: Domain randomization ranges applied during source-domain training in IsaacSim. $\mathcal{U}(a, b)$ denotes a uniform range; $\times$ denotes a multiplicative scale on the nominal value. Whole-body tracking and the locomotion family use the same randomization scheme with different magnitudes; the carried-object terms apply only to whole-body motion tracking.

Parameter	**Whole-Body Tracking**	**Locomotion & Loco-Manip.**
Dynamics randomization		
Ground friction	$\mathcal{U}(0.1, 2.0)$	$\mathcal{U}(0.1, 2.0)$
Restitution	$\mathcal{U}(0.0, 0.75)$	–
Base CoM shift (x)	$\mathcal{U}(-0.025, 0.025)$ m	$\mathcal{U}(-0.15, 0.15)$ m
Base CoM shift (y, z)	$\mathcal{U}(-0.05, 0.05)$ m	$\mathcal{U}(-0.15, 0.15)$ m
Added base mass	$\mathcal{U}(-2.0, 4.0)$ kg	$\mathcal{U}(-3.0, 6.0)$ kg
Link mass scale	$\mathcal{U}(0.9, 1.1)\times$	$\mathcal{U}(0.8, 1.3)\times$
DoF position bias	$\mathcal{U}(-0.01, 0.01)$ rad	$\mathcal{U}(-0.05, 0.05)$ rad
Actuation and control		
K_p scale	$\mathcal{U}(0.9, 1.1)\times$	$\mathcal{U}(0.8, 1.2)\times$
K_d scale	$\mathcal{U}(0.9, 1.1)\times$	$\mathcal{U}(0.8, 1.2)\times$
Torque RFI	$0.1\,\tau_{\max}$	$0.1\,\tau_{\max}$
Control delay	$\{0, 1\}$ steps	$\{0, 1\}$ steps
Perturbations		
Push interval	$\mathcal{U}(1, 3)$ s	$\mathcal{U}(5, 10)$ s
Max push velocity	0.5 m/s, 0.52 rad/s	$\mathcal{U}(0.1, 1.5)$
Carried-object randomization (whole-body tracking)		
Object friction	$\mathcal{U}(0.1, 0.6)$	
Object restitution	$\mathcal{U}(0.0, 1.0)$	
Object mass scale	$\mathcal{U}(1.0, 4.0)\times$	
Object inertia (I_{xx}) scale	$\mathcal{U}(0.5, 1.5)\times$	

The reported normalized velocity error is

$$\bar{E}_v = \frac{E_{\text{lin}} + E_{\text{ang}}}{E_{\text{lin}}^{\text{TFD}} + E_{\text{ang}}^{\text{TFD}}}, \tag{A.2}$$

computed per task under the same evaluation condition.

Whole-body tracking. For whole-body tracking, we report normalized mean per-joint position error,

$$E_{\text{mpjpe}} = \frac{1}{T|\mathscr{J}|} \sum_{t=1}^{T} \sum_{j \in \mathscr{J}} \left\| p_{t,j} - p_{t,j}^{\text{ref}} \right\|_2, \qquad \bar{E}_{\text{mpjpe}} = \frac{E_{\text{mpjpe}}}{E_{\text{mpjpe}}^{\text{TFD}}}, \tag{A.3}$$

where $p_{t,j}$ and $p_{t,j}^{\text{ref}}$ denote the deployed and reference joint or body-keypoint positions. For hardware whole-body tracking, a trial is also treated as failed if the robot falls before completing the motion.

Success rate. Slope traversal succeeds if the robot completes the specified trajectory without falling or leaving the ramp. Basket pulling succeeds if the basket crosses the finish line while the robot remains stable. For whole-body tracking tasks, completion additionally requires the robot not to fall during the reference motion.

B FADA Implementation Details

This appendix records the implementation choices used for FADA in the main experiments. We describe the deployable Planner–IDM architecture, source-domain pretraining, few-shot target adaptation, and the baselines used for comparison. The end-to-end procedure is summarized in Algorithm 1 of Section 4.

B.1 Planner–IDM Architecture

The deployable policy is the factorized student $\pi^s = (P_\phi, I_\psi)$ from Section 4. Both modules are transformer-based and operate on compact proprioceptive tokens rather than privileged simulator state. The planner receives the recent proprioceptive observation history and command and predicts a future proprioceptive chunk $\hat{Y}_t^K$. The IDM receives the same recent execution history together with a future chunk and predicts an action chunk $\hat{U}_t^K$. In all main experiments, the history length is H=30 and the prediction horizon is K=6. The planner and IDM encoder use 3-layer, 4-head transformers with hidden size 128, and the IDM decoder uses 2 transformer decoder layers with the same hidden size and number of heads.

The planner is implemented as a transformer encoder over observation-history and command-conditioned tokens. In implementation, the planner head predicts a residual future chunk relative to the latest history observation. We reconstruct the absolute future before passing it to the IDM,

$$\hat{Y}_t^K = o_t + \Delta\hat{Y}_t^K, \tag{B.1}$$

and use $\hat{Y}_t^K$ for the notation in the main text.

The IDM uses an encoder–decoder transformer interface. Its history token at each timestep is formed by adding the observation embedding and the executed-action embedding, followed by positional encoding. The planner similarly forms each history token by adding the observation embedding, a broadcast command embedding, and positional encoding. Future-state tokens are formed by embedding each future observation in the K-step chunk and adding future positional encodings. The IDM history encoder applies full self-attention over the history tokens. The decoder takes all future-state tokens as target tokens; since no causal mask is used, future tokens have full self-attention over the entire predicted future chunk, and each future token cross-attends to all encoded history tokens. The action head is applied to every decoded future token, producing the K-step action chunk in parallel. The IDM output is always a K-step action chunk. At deployment, the policy is executed in a receding-horizon manner: only the first action $\Pi_1(\hat{U}_t^K)$ is sent to the low-level controller, after which the history is updated and the planner and IDM are queried again. This is why the main IDM objective supervises the first action of the chunk, while full-chunk action supervision is studied as an ablation.

B.2 Source-Domain Training

As summarized in Section 4.1, student-rollout states are relabeled by restoring simulator snapshots and rolling the final oracle forward for K steps under the same command to obtain the oracle-shadow pair $(Y_{\text{orac}}^K, U_{\text{orac}}^K)$ and the relabeled action $a_t^\star$. Here we record the additional implementation details. Source-domain training first learns a privileged oracle policy π^o in IsaacSim with task rewards and domain randomization. The oracle has access to privileged simulator information during training, but the deployable student does not. The source student is trained with DAgger-style rollouts so that visited states can be paired with both rollout-policy trajectory data and privileged oracle labels.

Source buffers store both raw trajectory fields and oracle-shadow labels. For rollouts generated by the final oracle, these two signals coincide. For rollouts generated by the planner–IDM student

policy, we additionally store the realized trajectory (o_t, a_t, o_{t+1}) alongside the oracle-shadow relabel described above.

Training routes these stored targets according to their role. The planner updates use the oracle-shadow labels, so the predicted future is encouraged to be actionable by the final oracle through the stop-gradient IDM. IDM updates use action-observation pairs that are causally matched. Let $(Y^K_{\text{traj}}, U^K_{\text{traj}})$ denote the realized trajectory pair and $(Y^K_{\text{orac}}, U^K_{\text{orac}})$ denote the oracle-shadow pair. The IDM training pair is selected as

$$(Y^K_I, U^K_I) = \begin{cases} (Y^K_{\text{traj}}, U^K_{\text{traj}}), & \text{for trajectory-source batches,} \\ (Y^K_{\text{orac}}, U^K_{\text{orac}}), & \text{for oracle-source batches.} \end{cases} \tag{B.2}$$

The latter is also valid inverse-dynamics supervision because the oracle-shadow chunk is generated by rolling the final oracle forward in the shadow simulator, so its actions and future observations form a physically realized causal pair under the oracle policy. The IDM predicts an action chunk from $(\mathcal{O}^H_t, \mathcal{A}^H_t, Y^K_I)$, but the default loss supervises only $\Pi_1(U^K_I)$ to match receding-horizon deployment. In the separate-pass training used in our main runs, source optimization alternates two passes within each training iteration: an IDM pass trained with full teacher forcing on these matched future-action pairs, and a planner pass that keeps IDM weights fixed while optimizing the planner through the IDM action loss. Thus, oracle relabeling supplies intent labels, while trajectory fields preserve the realized dynamics needed by the IDM.

To broaden the IDM's coverage beyond near-optimal behavior, source data also includes suboptimal rollouts from intermediate oracle checkpoints. The suboptimal data budget is set to twice the amount of optimal data, and 20 intermediate oracle checkpoints are selected from the oracle training process. This additional data is used only to improve the diversity of source-domain inverse-dynamics supervision; all source rollouts still carry oracle labels for planner supervision.

B.3 Few-Shot Target Adaptation

Target adaptation uses only ordinary target rollouts collected by executing the source-trained policy in the fixed deployment condition ξ_{tgt}. No target rewards, privileged target labels, simulator calibration, or source-domain replay are used during the target update. Each target rollout is converted into the same window format as source IDM training: $(\mathcal{O}^H_t, \mathcal{A}^H_t, Y^K_{t,\text{exec}}, U^K_{t,\text{exec}})$. The future observations and action chunks come from the same physical or simulated rollout window, so they encode how the target system actually responds to executed actions.

During adaptation, we freeze the Planner P_ϕ and the pretrained IDM weights ψ, and optimize only the LoRA parameters $\Delta\psi$ inserted into the IDM. We train these parameters with the action loss in Eq. (4.5), using rank $r = 8$, scaling $\alpha = 16$, and dropout 0.05. These values were selected from a small grid over $r \in \{4, 8, 16\}$ and $\alpha \in \{8, 16, 32\}$. However, it should be noted that performance was not sensitive to these choices within the tested range. The resulting policy $(P_\phi, I_{\psi+\Delta\psi})$ keeps the same planner intent as the source policy but changes how that intent is translated into actions under ξ_{tgt}. This is the execution-only update tested throughout the main experiments. Unlike source-domain pretraining, target adaptation has no oracle relabels. Its supervision is entirely the paired target rollout data: the physically observed future $Y^K_{t,\text{exec}}$ and the executed action chunk $U^K_{t,\text{exec}}$. The same first-action inverse-dynamics target $\Pi_1(U^K_{t,\text{exec}})$ is used, so the adapted IDM is aligned with the action actually executed under receding-horizon deployment.

B.4 Baseline Implementations

We compare FADA against baselines that use the same source training pipeline, deployable observation space, and evaluation protocol whenever applicable. The purpose of these baselines is to isolate

whether few-shot target rollouts are most useful when they update the action-generation module rather than a monolithic student or a future-prediction objective. Their interfaces are visualized in Figure 5 (main text).

TF–DAgger. TF–DAgger is a transformer teacher–student policy trained from the same source-domain DAgger data as FADA. It maps recent observation-action history and the task command directly to the next action, without an explicit Planner–IDM factorization and without target-domain finetuning.

TF–CoPred-zs. TF–CoPred-zs is a zero-shot co-prediction baseline motivated by recent world-model and world-action-model approaches. It uses a shared transformer backbone to jointly predict the next deployable observation and action from history and command inputs, and is evaluated directly in the target condition.

TF–CoPred-ft. TF–CoPred-ft uses the same target rollout budget as FADA but adapts the shared co-prediction backbone with a future-observation prediction objective on target windows. This baseline tests whether target rollouts are sufficient when used to update a generic predictive representation, rather than the isolated IDM action-generation module.

All target-adapted methods are compared using the same target rollout budget and evaluation commands.

C Additional Analysis and Ablations

C.1 Architecture Ablations

This ablation examines the role of transformer-based token processing in the Planner–IDM interface. The experiment is run in the MuJoCo sim-to-sim setting on T1 Loco. + Payload, following the protocol of Appendix A.1, with 20 randomized-command trials and results normalized column-wise by the first row. LoRA vs. full IDM finetuning and the target rollout budget are studied in Section 5.5.

Transformer vs. MLP modules. Table 7 compares transformer modules against MLP replacements with the same input/output interface and a matched parameter budget on T1 Loco. + Payload. The IsaacSim Eval column measures source-domain evaluation, while the FADA column reports post-adaptation MuJoCo performance. The full transformer Planner–IDM gives the best source-domain evaluation and the best post-adaptation MuJoCo performance. Replacing more of the architecture with MLP modules steadily weakens target adaptation, suggesting that attention over history and future tokens is important for learning a reusable dynamics-alignment interface.

Table 7: Transformer vs. MLP Planner–IDM modules. We report $\bar{E}_v \downarrow$ on T1 Loco. + Payload. IsaacSim Eval denotes source-domain evaluation, and FADA denotes post-adaptation MuJoCo evaluation. Codes denote planner / IDM encoder / IDM decoder, where T is transformer and M is MLP. MLP variants use the same inputs, outputs, and parameter budget as the transformer model.

Code	IsaacSim Eval	FADA
MMM	$1.0000_{\pm 0.0915}$	$1.0000_{\pm 0.1235}$
MTM	$0.9729_{\pm 0.0904}$	$0.8345_{\pm 0.1129}$
TTM	$0.9742_{\pm 0.0851}$	$0.7664_{\pm 0.1210}$
TTT (ours)	$\mathbf{0.9622}_{\pm 0.0849}$	$\mathbf{0.7056}_{\pm 0.1150}$

C.2 Same-Simulator Domain Shift Analysis

Same-simulator shifts isolate dynamics mismatch while avoiding additional confounders from simulator implementation or hardware deployment. In this setting, we vary target payload or terrain conditions while keeping the simulator implementation, observation conventions, and reset procedures fixed. Table 8 reports IsaacSim-to-IsaacSim transfer results under held-out target conditions. Each policy is evaluated with 1024 parallel IsaacSim environments, enabling a statistically more reliable comparison than smaller-scale hardware trials. Since source and target share the same simulator implementation, the domain gap is smaller than in MuJoCo or real-world deployment. Nevertheless, FADA consistently improves over zero-shot transfer across payload and terrain shifts.

Table 8: IsaacSim-to-IsaacSim held-out domain shifts. We report $\bar{E}_v \downarrow$ using 1024 parallel IsaacSim evaluation environments. Even when source and target share the same simulator implementation, FADA improves over zero-shot transfer on held-out dynamics.

Method	G1 Loco. + Payload	T1 Slope Traversal	T1 Loco. + Payload
TF–DAgger	$1.0000_{\pm 0.0444}$	$1.0000_{\pm 0.0418}$	$1.0000_{\pm 0.0278}$
FADA-zs	$1.0501_{\pm 0.0234}$	$0.9906_{\pm 0.0351}$	$1.0177_{\pm 0.0189}$
FADA (ours)	$\mathbf{0.8514}_{\pm 0.0282}$	$\mathbf{0.9461}_{\pm 0.0243}$	$\mathbf{0.9380}_{\pm 0.0215}$

D Additional Sim-to-Real Results

In addition to the hardware results reported in the main text, we provide qualitative rollouts across a broader set of deployment conditions in Figure 13. These trials compare FADA against the zero-shot variant FADA-zs and the TF–DAgger baseline on contact-rich terrain, payload transport, and whole-body tracking tasks. Across settings, the same trend appears: zero-shot policies often produce partially correct motion but lose tracking under the target dynamics, while IDM adaptation improves the execution needed to realize the commanded behavior.

Figure 13 shows seven representative examples. On G1 slope walking, FADA maintains progress along the plank, whereas the baselines drift or step off the traversable region. On soft-mat and sand-terrain Kungfu tracking, adaptation improves posture stability and keeps the robot closer to the commanded tracking region despite deformable contact. For payload tasks, including grocery carrying, dance with payload, and T1 circle walking with payload, FADA better compensates for load-induced changes in balance and body velocity, reducing drift and improving path following. In the T1 laundry-basket dragging task, the adapted policy maintains a straighter pulling direction and reaches the finish region, while the baselines fail to sustain the required contact-rich motion.

These examples highlight two recurring failure modes of zero-shot transfer. First, under terrain or contact mismatch, the policy may still initiate the correct skill but accumulate foot-placement and posture errors over time. Second, under payload or dragging forces, the planned motion remains reasonable, but the action mapping is no longer sufficient to realize the intended body motion. IDM finetuning addresses these failures by recalibrating the plan-to-action map from target rollouts, without changing the planner or requiring task-specific rewards.

Figure 13: Additional qualitative sim-to-real deployments. We compare FADA, FADA-zs, and TF–DAgger across seven hardware settings: **(a)** G1 slope walking, **(b)** G1 Kungfu tracking on soft mats, **(c)** G1 grocery carrying through poles, **(d)** T1 laundry-basket dragging, **(e)** G1 dance with payload, **(f)** G1 Kungfu tracking on sand, and **(g)** T1 circle walking with payload. Green annotations indicate successful tracking, stable regions, or completed paths; red annotations indicate drift, instability, or task failure. Across tasks, IDM adaptation improves execution under terrain, payload, and contact mismatch.